

Курсовая работа

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

**«САРАТОВСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИМЕНИ Н. Г. ЧЕРНЫШЕВСКОГО»**

Кафедра математической кибернетики и компьютерных наук

**СРАВНЕНИЕ АЛГОРИТМОВ СОРТИРОВКИ: БЫСТРАЯ
СОРТИРОВКА VS СОРТИРОВКА СЛИЯНИЕМ**

КУРСОВАЯ РАБОТА

студента 1 курса 111 группы

направления 02.03.02 — Фундаментальная информатика и информационные
технологии

факультета КНиИТ

Попов Даниил Евгеньевич

Научный руководитель

доцент, к. ф.-м. н.

Г. Г. Наркайтис

Заведующий кафедрой

доцент, к. ф.-м. н.

С. В. Миронов

Саратов 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 Теоретические основы алгоритмов сортировки	4
1.1 Быстрая сортировка	4
1.2 Сортировка слиянием	4
1.3 Теоретическое сравнение сложности	5
2 Реализация алгоритмов	6
2.1 Быстрая сортировка на C++	6
2.2 Сортировка слиянием на Python	6
3 Сравнительный анализ	8
3.1 Эмпирическое время выполнения	8
3.2 Использование памяти	8
3.3 Стабильность	9
3.4 Итоговое сравнение	9
ЗАКЛЮЧЕНИЕ	10
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	11
Приложение А Исходный код быстрой сортировки на C++	12
Приложение Б Исходный код сортировки слиянием на Python	14

ВВЕДЕНИЕ

Задача упорядочивания данных является одной из фундаментальных в области информатики и программирования. Алгоритмы сортировки применяются повсеместно— от баз данных и операционных систем до компиляторов и графических движков. Эффективность выбранного алгоритма напрямую влияет на производительность программной системы в целом.

Среди множества существующих алгоритмов сортировки особое место занимают два: быстрая сортировка (*Quicksort*), предложенная Ч. А. Р. Хоаром в 1962 году [1], и сортировка слиянием (*Merge sort*), впервые описанная Джоном фон Нейманом в 1945 году. Оба алгоритма относятся к классу «разделяй и властвуй» и широко используются на практике.

Цель работы— провести теоретический и практический анализ алгоритмов быстрой сортировки и сортировки слиянием, выявить их сильные и слабые стороны, а также определить условия предпочтительного применения каждого из них.

Для достижения поставленной цели необходимо решить следующие задачи:

1. изучить теоретические основы алгоритмов быстрой сортировки и сортировки слиянием;
2. реализовать оба алгоритма на языках программирования C++ и Python;
3. провести сравнительный анализ временной сложности и использования памяти;
4. сформулировать рекомендации по выбору алгоритма.

1 Теоретические основы алгоритмов сортировки

1.1 Быстрая сортировка

Быстрая сортировка— рекурсивный алгоритм, основанный на стратегии «разделяй и властвуй» [2]. Алгоритм выбирает из массива элемент, называемый *опорным* (pivot), и переставляет элементы таким образом, чтобы все элементы, меньшие опорного, оказались левее него, а большие— правее. После этого процедура рекурсивно применяется к каждой из двух получившихся частей.

Временная сложность быстрой сортировки определяется следующим образом. В **среднем и лучшем** случае рекуррентное соотношение имеет вид:

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n), \quad (1.1)$$

решение которого по мастер-теореме даёт $T(n) = \Theta(n \log n)$.

В **худшем** случае (отсортированный или обратнo-отсортированный массив при выборе крайнего элемента в качестве опорного) рекуррентное соотношение принимает вид:

$$T(n) = T(n - 1) + \Theta(n) = \Theta(n^2). \quad (1.2)$$

Быстрая сортировка является сортировкой **на месте**: дополнительной памяти под данные не требуется, хотя стек рекурсии занимает $O(\log n)$ в среднем и $O(n)$ в худшем случае [3].

1.2 Сортировка слиянием

Сортировка слиянием— также рекурсивный алгоритм класса «разделяй и властвуй» [4]. Массив делится пополам, каждая половина рекурсивно сортируется, после чего две отсортированные половины *сливаются* в один упорядоченный массив.

Рекуррентное соотношение для сортировки слиянием одинаково во всех случаях:

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n) = \Theta(n \log n). \quad (1.3)$$

В отличие от быстрой сортировки, сортировка слиянием требует дополнительной памяти объёмом $O(n)$ для хранения вспомогательных подмассивов на этапе слияния [5].

1.3 Теоретическое сравнение сложности

В таблице 1.1 приведено сводное сравнение временной и пространственной сложности рассматриваемых алгоритмов.

Таблица 1.1 – Сравнение сложности алгоритмов сортировки

Алгоритм	Лучший	Средний	Худший	Память
Быстрая сортировка	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
Сортировка слиянием	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$

На рисунке 1.1 показана графическая интерпретация роста функций $O(n \log n)$ и $O(n^2)$ в зависимости от размера входных данных.

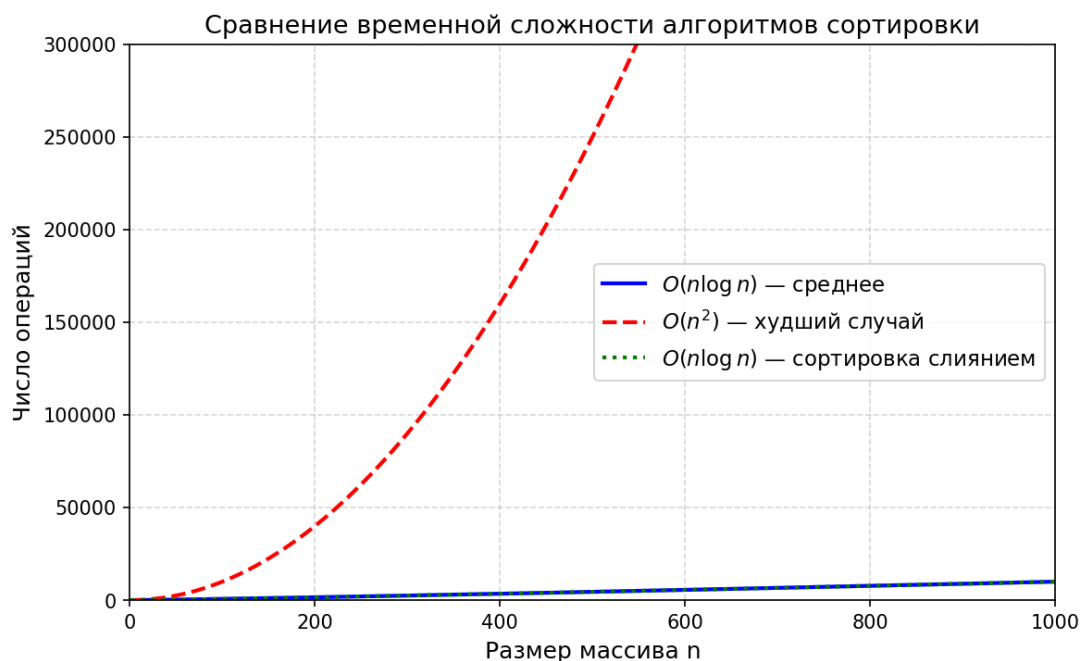


Рисунок 1.1 – Сравнение временной сложности алгоритмов сортировки

Как видно из рисунка 1.1, при небольших значениях n разница незначительна, однако с ростом размера массива $O(n^2)$ резко превосходит $O(n \log n)$.

2 Реализация алгоритмов

2.1 Быстрая сортировка на C++

Реализация быстрой сортировки выполнена на языке C++ с использованием стандартной библиотеки `<vector>` и `<chrono>` для замера времени выполнения. Полный исходный код приведён в приложении А.

Ключевой функцией является `partition`, которая разделяет подмассив `arr[low..high]` относительно опорного элемента `arr[high]` и возвращает его итоговый индекс. Ниже приведён фрагмент основной рекурсивной функции:

```
1 void quickSort(std::vector<int>& arr, int low, int high) {
2     if (low < high) {
3         int pi = partition(arr, low, high);
4         quickSort(arr, low, pi - 1);
5         quickSort(arr, pi + 1, high);
6     }
7 }
```

Функция `quickSort` принимает ссылку на вектор `arr`, а также границы текущего подмассива— `low` и `high`. Рекурсия завершается, когда `low >= high`, то есть подмассив содержит не более одного элемента [6].

2.2 Сортировка слиянием на Python

Реализация сортировки слиянием выполнена на языке Python 3. Полный исходный код приведён в приложении Б.

Алгоритм состоит из двух функций. Функция `merge_sort` рекурсивно делит массив пополам, а функция `merge` выполняет слияние двух отсортированных подмассивов. Ниже приведён фрагмент функции слияния:

```
1 def merge(arr: list, left: int, mid: int, right: int) -> None:
2     left_part = arr[left:mid + 1]
3     right_part = arr[mid + 1:right + 1]
4     i = j = 0
5     k = left
6     while i < len(left_part) and j < len(right_part):
7         if left_part[i] <= right_part[j]:
8             arr[k] = left_part[i]; i += 1
9         else:
```

```
10         arr[k] = right_part[j]; j += 1
11     k += 1
```

Переменные i , j и k являются индексами левого подмассива, правого подмассива и результирующего массива соответственно. Стабильность алгоритма обеспечивается строгим неравенством $\text{left_part}[i] < \text{right_part}[j]$ [7].

3 Сравнительный анализ

3.1 Эмпирическое время выполнения

Для оценки практической производительности оба алгоритма были запущены на случайных массивах целых чисел различного размера. Каждый эксперимент повторялся пять раз, результатом являлось среднее время выполнения. На рисунке 3.1 представлены результаты замеров.

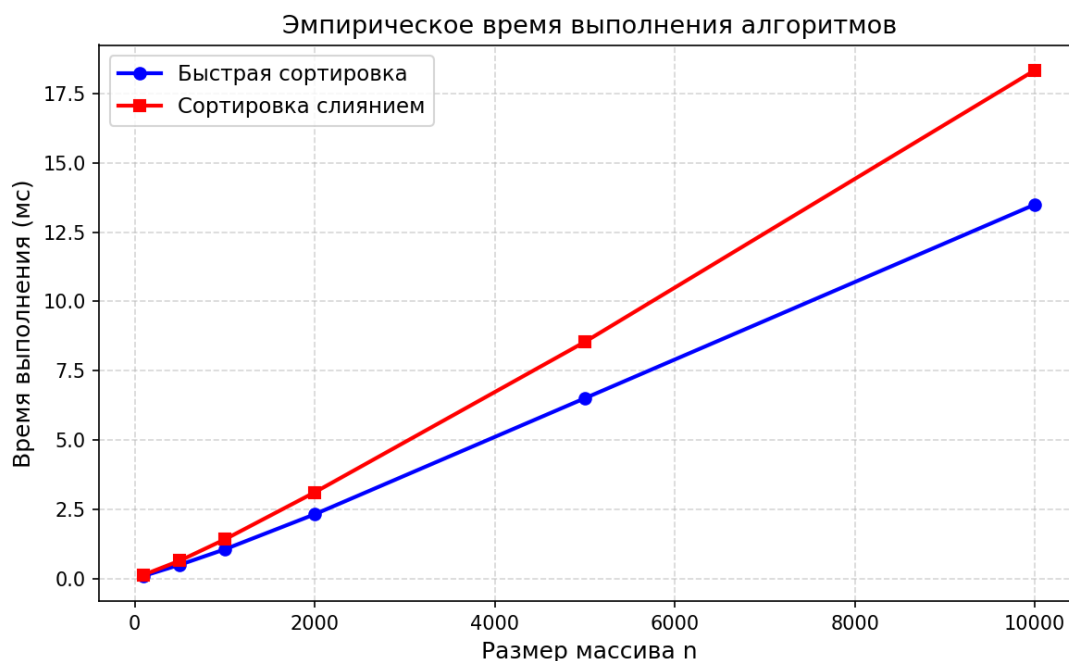


Рисунок 3.1 – Эмпирическое время выполнения алгоритмов сортировки

Из рисунка 3.1 следует, что на случайных данных быстрая сортировка незначительно опережает сортировку слиянием за счёт лучшей кэш-локальности: операции `swap` производятся над элементами самого массива, тогда как `merge` создаёт вспомогательные подмассивы и копирует данные.

3.2 Использование памяти

Принципиальное различие алгоритмов — потребление дополнительной памяти. Быстрая сортировка работает *на месте*: единственные накладные расходы — стек вызовов глубиной $O(\log n)$ в среднем случае. Сортировка слиянием на каждом уровне рекурсии создаёт вспомогательные массивы суммарным объёмом $O(n)$ [2].

Таким образом, для массива из n элементов типа `int` (4 байта) суммарный дополнительный расход памяти сортировки слиянием составит приблизительно:

$$M(n) = 4n \text{ байт} = \frac{n}{256} \text{ КБ.} \quad (3.1)$$

3.3 Стабильность

Сортировка слиянием является **стабильной**: элементы с одинаковыми ключами сохраняют исходный относительный порядок, что важно, например, при многоуровневой сортировке таблиц [5].

Классическая реализация быстрой сортировки **нестабильна**: функция `partition` может нарушать исходный порядок равных элементов.

3.4 Итоговое сравнение

В таблице 3.1 приведено итоговое сравнение характеристик рассматриваемых алгоритмов.

Таблица 3.1 – Итоговое сравнение алгоритмов

Характеристика	Быстрая сортировка	Сортировка слиянием
Среднее время	$O(n \log n)$	$O(n \log n)$
Худшее время	$O(n^2)$	$O(n \log n)$
Доп. память	$O(\log n)$	$O(n)$
Стабильность	Нет	Да
Кэш-локальность	Высокая	Средняя
Практическая скорость	Выше на случайных данных	Стабильна на любых данных

ЗАКЛЮЧЕНИЕ

В ходе выполнения данной курсовой работы были изучены и реализованы два классических алгоритма сортировки— быстрая сортировка и сортировка слиянием.

В результате проведённого теоретического анализа установлено, что оба алгоритма имеют одинаковую среднюю временную сложность $O(n \log n)$, однако существенно различаются в поведении при неблагоприятных входных данных: быстрая сортировка деградирует до $O(n^2)$, тогда как сортировка слиянием гарантирует $O(n \log n)$ при любом наборе данных.

Эмпирические эксперименты подтвердили теоретические выводы: на случайных данных быстрая сортировка незначительно быстрее за счёт лучшей кэш-локальности, однако уступает по предсказуемости поведения.

На основе проведённого анализа можно сформулировать следующие практические рекомендации:

- если память ограничена, а данные случайны— предпочтительна быстрая сортировка;
- если требуется гарантированное время $O(n \log n)$ или стабильность— предпочтительна сортировка слиянием;
- в стандартных библиотеках (например, `std::stable_sort` в C++) применяется именно сортировка слиянием [6].

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1 Hoare, Charles Antony Richard. Quicksort [Text] // The Computer Journal. — 1962. — Vol. 5, no. 1. — P. 10–16.
- 2 Алгоритмы: построение и анализ [Текст] / Кормен, Томас, Лейзерсон, Чарльз, Ривест, Рональд и Штайн, Клиффорд. — Москва : Издательский дом Вильямс, 2009.
- 3 Седжвик, Роберт. Алгоритмы на C++ [Текст]. — Москва : Издательский дом Вильямс, 2011.
- 4 Кнут, Дональд Эрвин. Искусство программирования. Том 3: Сортировка и поиск [Текст]. — Москва : Издательский дом Вильямс, 1998.
- 5 Петров, А. В. Анализ алгоритмов сортировки и их применение [Текст] / Петров, А. В. и Сидоров, К. Н. // Вестник компьютерных и информационных технологий. — 2018. — Т. 3. — С. 14–21.
- 6 `std::sort` — [cppreference.com](https://en.cppreference.com/w/cpp/algorithm/sort) [Электронный ресурс онлайн]. — [Б. м. : б. и.]. — Режим доступа: <https://en.cppreference.com/w/cpp/algorithm/sort> (дата обращения: 10.05.2025). Загл. с экр. Яз. англ.
- 7 Сортировка — Python 3 Documentation [Электронный ресурс онлайн]. — [Б. м. : б. и.]. — Режим доступа: <https://docs.python.org/3/howto/sorting.html> (дата обращения: 10.05.2025). Загл. с экр. Яз. англ.

ПРИЛОЖЕНИЕ А

Исходный код быстрой сортировки на C++

```
1  #include <iostream>
2  #include <vector>
3  #include <chrono>
4  #include <algorithm>
5
6  // Разделение массива относительно опорного элемента.
7  // Возвращает индекс опорного элемента после разделения.
8  int partition(std::vector<int>& arr, int low, int high) {
9      int pivot = arr[high]; // опорный элемент
10     int i = low - 1;
11
12     for (int j = low; j < high; j++) {
13         if (arr[j] <= pivot) {
14             i++;
15             std::swap(arr[i], arr[j]);
16         }
17     }
18     std::swap(arr[i + 1], arr[high]);
19     return i + 1;
20 }
21
22 // Рекурсивная быстрая сортировка.
23 // arr --- сортируемый массив, low --- левая граница, high --- правая граница.
24 void quickSort(std::vector<int>& arr, int low, int high) {
25     if (low < high) {
26         int pi = partition(arr, low, high);
27         quickSort(arr, low, pi - 1);
28         quickSort(arr, pi + 1, high);
29     }
30 }
31
32 int main() {
33     std::vector<int> arr = {64, 34, 25, 12, 22, 11, 90, 5, 77, 43};
34
35     std::cout << "До сортировки: ";
36     for (int x : arr) std::cout << x << " ";
37     std::cout << "\n";
38
39     auto start = std::chrono::high_resolution_clock::now();
40     quickSort(arr, 0, arr.size() - 1);
```

```
41     auto end = std::chrono::high_resolution_clock::now();
42
43     std::cout << "После сортировки: ";
44     for (int x : arr) std::cout << x << " ";
45     std::cout << "\n ";
46
47     auto duration = std::chrono::duration_cast<std::chrono::microseconds>(end
    ↪ - start);
48     std::cout << "Время выполнения: " << duration.count() << " мкс \n ";
49
50     return 0;
51 }
```

ПРИЛОЖЕНИЕ Б

Исходный код сортировки слиянием на Python

```
1 import time
2 import random
3
4
5 def merge(arr: list, left: int, mid: int, right: int) -> None:
6     """Слияние двух отсортированных подмассивов arr[left..mid] и
       ↪ arr[mid+1..right]."""
7     left_part = arr[left:mid + 1]
8     right_part = arr[mid + 1:right + 1]
9
10    i = 0 # индекс для left_part
11    j = 0 # индекс для right_part
12    k = left # индекс для результирующего массива
13
14    while i < len(left_part) and j < len(right_part):
15        if left_part[i] <= right_part[j]:
16            arr[k] = left_part[i]
17            i += 1
18        else:
19            arr[k] = right_part[j]
20            j += 1
21        k += 1
22
23    while i < len(left_part):
24        arr[k] = left_part[i]
25        i += 1
26        k += 1
27
28    while j < len(right_part):
29        arr[k] = right_part[j]
30        j += 1
31        k += 1
32
33
34 def merge_sort(arr: list, left: int, right: int) -> None:
35     """Рекурсивная сортировка слиянием.
36
37     arr --- сортируемый список,
38     left --- левая граница подмассива,
39     right --- правая граница подмассива.
```

```
40     """
41     if left < right:
42         mid = (left + right) // 2
43         merge_sort(arr, left, mid)
44         merge_sort(arr, mid + 1, right)
45         merge(arr, left, mid, right)
46
47
48 if __name__ == "__main__":
49     arr = [64, 34, 25, 12, 22, 11, 90, 5, 77, 43]
50
51     print("До сортировки:", arr)
52
53     start = time.perf_counter()
54     merge_sort(arr, 0, len(arr) - 1)
55     end = time.perf_counter()
56
57     print("После сортировки:", arr)
58     print(f"Время выполнения: {(end - start) * 1e6:.2f} мкс")
```
